

DevOps Security

IE University - BCSAI - SDD - 2025

Agenda

1. Introduction to DevOps Security
2. Understanding DevSecOps
3. Security in CI/CD Pipelines
4. Infrastructure Security
5. Secrets Management
6. Monitoring and Incident Response
7. Security Automation and Compliance

1. Introduction

The challenge: DevOps enables rapid deployments (thousands per day), but traditional security approaches can't keep pace.

Traditional approach:

- Security as a final checkpoint before production
- "Security gate" or "penetration testing phase"
- Creates bottlenecks and delays releases

The cost of breaches:

- Average breach cost: **\$4.45 million** (IBM 2023)
- Organizations without security automation face higher costs and longer detection times

DevOps Security: The Solution

DevSecOps = Development + Security + Operations

Core principle: Security is not an afterthought, it's integrated throughout the entire software development lifecycle.

Key benefits:

- Early vulnerability detection
- Reduced remediation costs
- Maintained development velocity
- Regulatory compliance (GDPR, HIPAA, PCI-DSS)

2. What is DevSecOps?

Security becomes a **shared responsibility** throughout the software development lifecycle.

Core principles:

- **Automation First:** Security checks must be automated
- **Shared Responsibility:** Everyone owns security
- **Continuous Security:** Ongoing assessments, not discrete events
- **Collaboration:** Security teams guide, not gatekeep

Traditional vs DevSecOps Model

Aspect	Traditional	DevSecOps
Timing	End of lifecycle	Throughout lifecycle
Responsibility	Security team only	Entire team
Process	Manual reviews	Automated scanning
Speed	Bottleneck	Maintains velocity
Cost	High remediation	Low early fixes

Shift-Left Security

Concept: Move security activities **earlier** in the development lifecycle.

Why it matters:

- Design phase fix: **\$100**
- Development phase: **\$1,000**
- Testing phase: **\$10,000**
- Production: **\$100,000+**

Key practices:

- Security requirements during planning
- Threat modeling during design
- Secure coding standards
- Developer security training

3. Security in CI/CD Pipelines

Goal: Every code change undergoes security validation before production.

Security testing types:

- **SAST:** Static Application Security Testing
- **DAST:** Dynamic Application Security Testing
- **SCA:** Software Composition Analysis

```
Code Commit → SAST → Build → DAST → SCA → Deploy → Monitor
```


Static Application Security Testing (SAST)

What it does: Analyzes source code to identify vulnerabilities **without** executing the application.

Detects:

- SQL injection
- Cross-site scripting (XSS)
- Buffer overflows
- Insecure cryptographic implementations

When: On every push/pull request (before build)

SAST: Vulnerable vs Secure Code

```
# Vulnerable: SQL Injection
def get_user(username):
    query = f"SELECT * FROM users WHERE username = '{username}'"
    return db.execute(query)

# Secure: Parameterized query
def get_user_secure(username):
    query = "SELECT * FROM users WHERE username = %s"
    return db.execute(query, (username,))
```

SAST in GitHub Actions

```
name: Security Pipeline
on:
  push:
    branches: [main, develop]

jobs:
  sast-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run Semgrep SAST
        uses: returntocorp/semgrep-action@v1
        with:
          config: >-
            p/security-audit
            p/owasp-top-ten
```

Dynamic Application Security Testing (DAST)

What it does: Tests running applications by simulating attacks from external perspective.

Detects:

- Authentication flaws
- Session management issues
- Server misconfigurations
- Input validation vulnerabilities

When: After deployment to staging environment

Tool example: OWASP ZAP

Software Composition Analysis (SCA)

What it does: Scans **third-party dependencies** for known vulnerabilities.

Why it matters:

- Modern apps: 80-90% third-party code
- Dependencies inherit vulnerabilities
- License compliance issues

Tools: Snyk, Dependabot, OWASP Dependency-Check

SCA in GitHub Actions

```
sca-scan:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Run Snyk vulnerability scan
      uses: snyk/actions/node@master
      env:
        SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
      with:
        args: --severity-threshold=high
```

4. Infrastructure Security

Modern infrastructure challenges:

- Infrastructure as Code (IaC) misconfigurations
- Container vulnerabilities
- Kubernetes complexity

Key areas:

- IaC Security
- Container Security
- Kubernetes Security

Infrastructure as Code Security

Common IaC security issues:

- Overly permissive IAM policies
- Publicly accessible storage buckets
- Unencrypted data
- Missing network security controls
- Hardcoded credentials

Tools: Checkov, tfsec, KICS

IaC: Insecure vs Secure

```
# Insecure S3 bucket
resource "aws_s3_bucket" "data" {
  bucket = "sensitive-data"
  acl    = "public-read"  # BAD!
}

# Secure S3 bucket
resource "aws_s3_bucket" "data_secure" {
  bucket = "sensitive-data"
}

resource "aws_s3_bucket_public_access_block" "secure" {
  bucket                  = aws_s3_bucket.data_secure.id
  block_public_acls       = true
  block_public_policy     = true
  restrict_public_buckets = true
}
```

Container Security Best Practices

Dockerfile security:

```
# Use specific version tags (not 'latest')
FROM python:3.11-slim-bookworm

# Create non-root user
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY --chown=appuser:appgroup . .

# Run as non-root
USER appuser

ENTRYPOINT ["python", "app.py"]
```

Container Security Checklist

Practice	Why
Specific image tags	Avoid unexpected changes
Non-root user	Limit container privileges
Minimal base images	Reduce attack surface
Image scanning	Detect vulnerabilities
No secrets in images	Prevent credential exposure
Health checks	Enable orchestration

Tool: Trivy for container image scanning

Kubernetes Security

Security layers:

1. **Cluster hardening:** API server, etcd encryption
2. **Workload policies:** Pod security contexts
3. **Network controls:** Network policies
4. **Runtime protection:** Admission controllers

Secure Pod Configuration

```
apiVersion: v1
kind: Pod
metadata:
  name: secure-app
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
  containers:
  - name: app
    image: myapp:v1.0.0
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
    resources:
      limits:
        cpu: "500m"
        memory: "256Mi"
```

Kubernetes Network Policy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-network-policy
spec:
  podSelector:
    matchLabels:
      app: api
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 8080
```

Principle: Only allow necessary communication between pods.

5. Secrets Management

Problem: Sensitive credentials (API keys, passwords, certificates) must be securely stored, distributed, and rotated.

Improper secrets handling = one of the most common breach causes.

Secrets Management Principles

Principle	Implementation
Never in code	Use secrets managers
Encrypt at rest	KMS, Vault
Encrypt in transit	TLS everywhere
Least privilege	Minimum access needed
Audit logging	Track all access
Regular rotation	Automate rotation

HashiCorp Vault Example

```
import hvac
import os

class SecretsManager:
    def __init__(self):
        self.client = hvac.Client(
            url=os.environ['VAULT_ADDR'],
            token=os.environ['VAULT_TOKEN']
        )

    def get_database_credentials(self):
        secret = self.client.secrets.kv.v2.read_secret_version(
            path='database/production',
            mount_point='secret'
        )
        return secret['data']['data']
```

Kubernetes External Secrets

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: database-credentials
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: vault-backend
    kind: SecretStore
  target:
    name: database-secret
  data:
    - secretKey: username
      remoteRef:
        key: secret/data/database/production
        property: username
```

6. Security Monitoring

Key components:

- **Centralized Logging:** Aggregate all logs
- **SIEM:** Security Information and Event Management
- **IDS:** Intrusion Detection Systems
- **RASP:** Runtime Application Self-Protection

Goal: Rapid detection and response to security incidents.

Security Event Logging

```
import logging
import json
from datetime import datetime

class SecurityLogger:
    def log_authentication_event(self, user_id, success, ip):
        event = {
            'timestamp': datetime.utcnow().isoformat(),
            'event_type': 'authentication',
            'user_id': user_id,
            'success': success,
            'source_ip': ip
        }
        self.logger.info(json.dumps(event))

    def log_authorization_failure(self, user_id, resource):
        event = {
            'timestamp': datetime.utcnow().isoformat(),
            'event_type': 'authorization_failure',
            'user_id': user_id,
            'resource': resource
        }
        self.logger.warning(json.dumps(event))
```

Security Alerts with Prometheus

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: security-alerts
spec:
  groups:
  - name: security
    rules:
    - alert: HighFailedLoginAttempts
      expr: |
        sum(rate(authentication_failures_total[5m]))
        by (source_ip) > 10
      for: 2m
      labels:
        severity: critical
      annotations:
        summary: "High rate of failed login attempts"
```

7. Security Automation & Compliance

Challenge: Meet regulatory requirements (GDPR, HIPAA, PCI-DSS) without sacrificing agility.

Solution: Policy as Code

- Define policies in code
- Automate enforcement
- Continuous compliance scanning
- Audit trail generation

Open Policy Agent (OPA)

```
package kubernetes.admission

# Deny pods running as root
deny[msg] {
    input.request.kind.kind == "Pod"
    container := input.request.object.spec.containers[_]
    not container.securityContext.runAsNonRoot
    msg := sprintf("Container %v must run as non-root",
                    [container.name])
}

# Deny privileged containers
deny[msg] {
    input.request.kind.kind == "Pod"
    container := input.request.object.spec.containers[_]
    container.securityContext.privileged
    msg := sprintf("Container %v cannot be privileged",
                    [container.name])
}
```

Compliance Scanning Pipeline

```
compliance-scan:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Run compliance checks
      uses: aquasecurity/trivy-action@master
      with:
        scan-type: 'config'
        scan-ref: '.'
        format: 'json'
        output: 'compliance-report.json'

    - name: Generate compliance report
      run: |
        python scripts/generate_compliance_report.py \
          --framework CIS \
          --output compliance-summary.pdf
```


Summary: DevSecOps Key Practices

Phase	Security Practice
Plan	Security requirements, threat modeling
Code	Secure coding, SAST
Build	SCA, dependency scanning
Test	DAST, penetration testing
Deploy	IaC scanning, container scanning
Operate	Monitoring, incident response
Monitor	SIEM, alerting, compliance

DevSecOps Cultural Shift

Security is everyone's responsibility:

- Developers: Write secure code, fix vulnerabilities
- Operations: Secure infrastructure, monitor threats
- Security: Provide guidance, tooling, training

Investment areas:

- Developer security training
- Automated security tooling
- Clear security expectations
- Self-service vulnerability fixes

Key Takeaways

1. **Shift security left:** Earlier detection = lower costs
2. **Automate security testing:** SAST, DAST, SCA in CI/CD
3. **Secure infrastructure as code:** Scan IaC and containers
4. **Manage secrets properly:** Never in code, always encrypted
5. **Monitor continuously:** Detect and respond quickly
6. **Policy as code:** Automate compliance enforcement
7. **Shared responsibility:** Security is everyone's job

Getting Started with DevSecOps

Priority order for implementation:

1. **Add SAST/SCA** to CI/CD pipeline
2. **Implement secrets management** (Vault, cloud secrets)
3. **Enable security monitoring** and alerting
4. **Scan container images** before deployment
5. **Add DAST** for staging environments
6. **Implement policy-as-code** enforcement
7. **Continuous improvement:** adapt to new threats

Resources:

- OWASP Top Ten: owasp.org/www-project-top-ten
- CIS Benchmarks: cisecurity.org
- NIST Container Security Guide: SP 800-190